JENKINS_TASK-3

Task 1

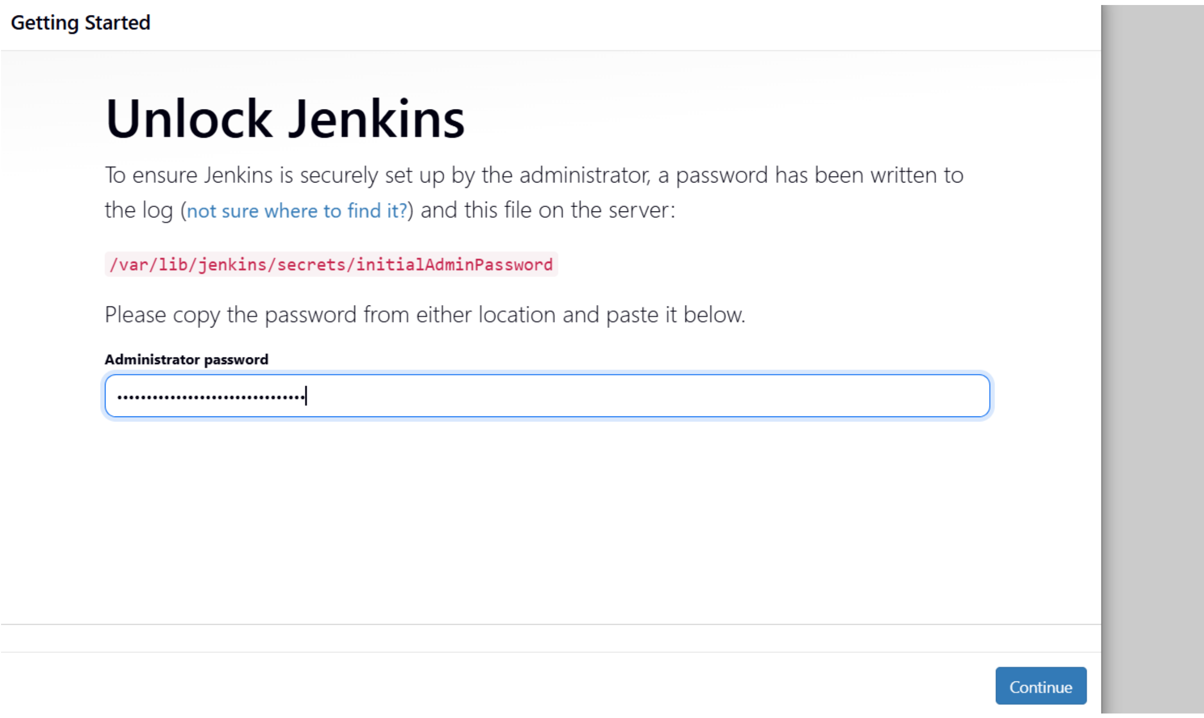
Create one Jenkins job using the below code and create three stages:

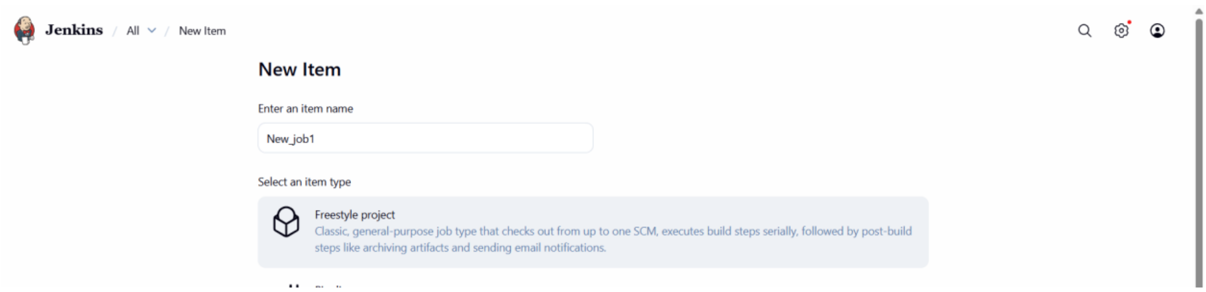
- **stage1: Git clone to download the source code.**
- **stage2: Sonarqube Integration to check the quality of code.**
- **stage3: Slack Integration to send the alerts to slack.**

URL: https://github.com/betawins/VProfile-1.git

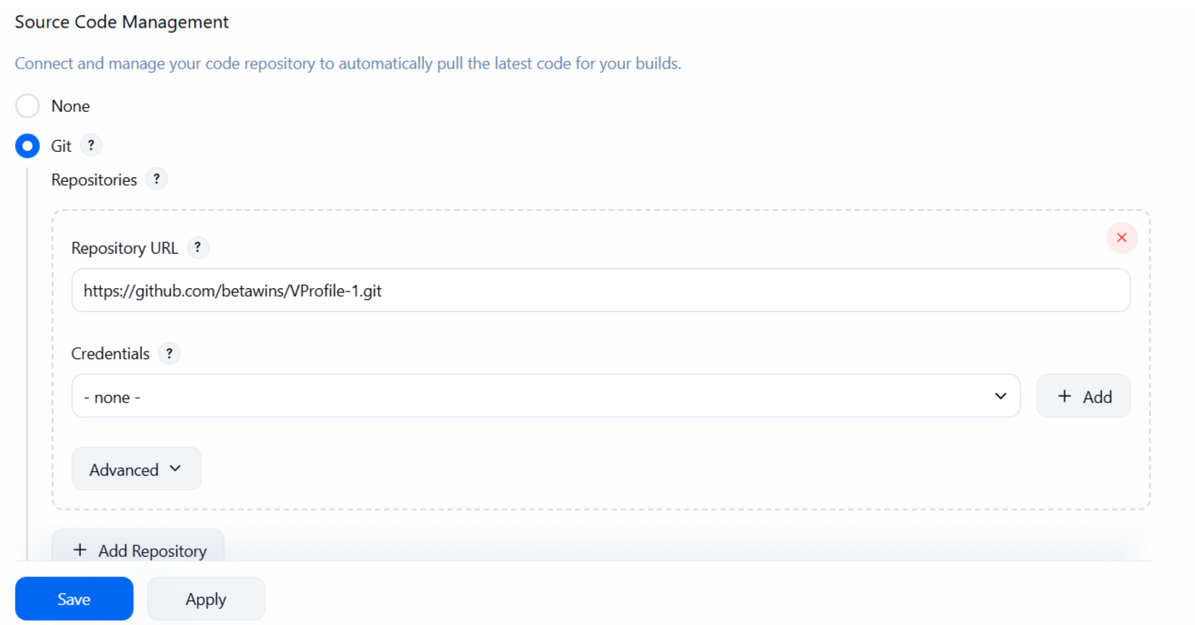
Stage:1

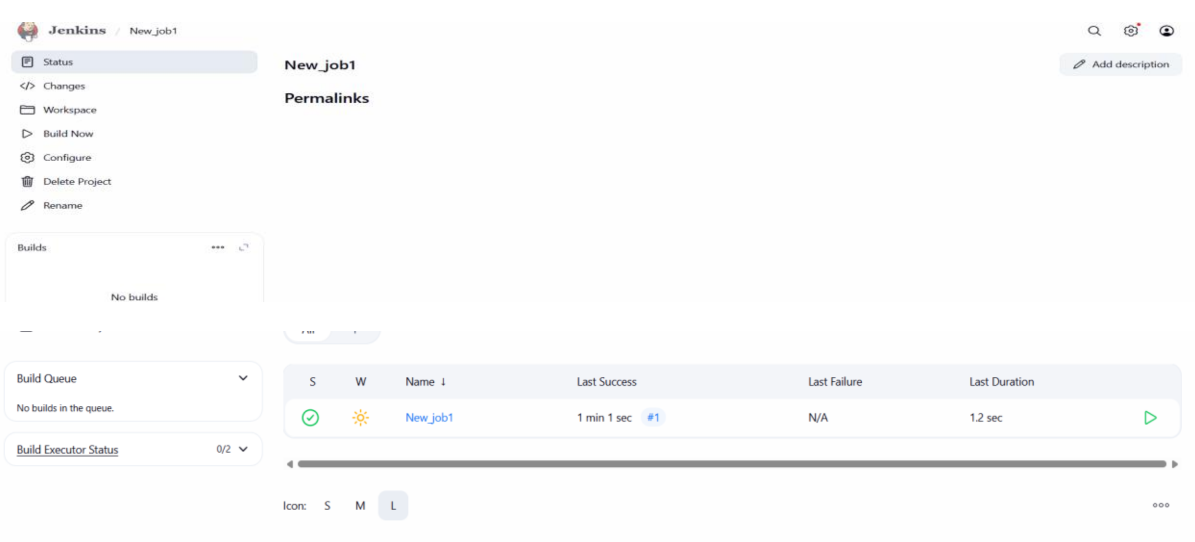
- create an instance with 20gb and t2 large
- launch instance and allow port number 8080 in SG
- connect with instance and install java and Jenkins



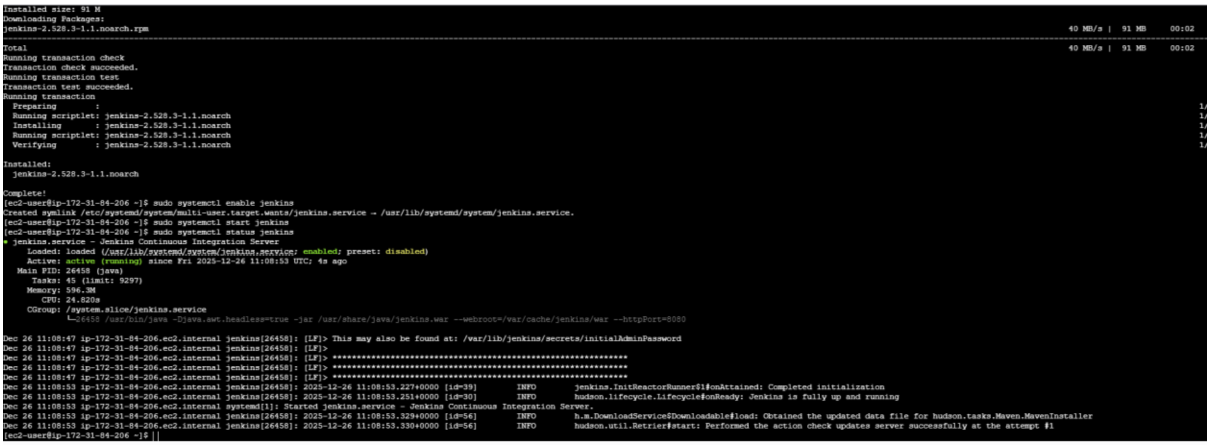


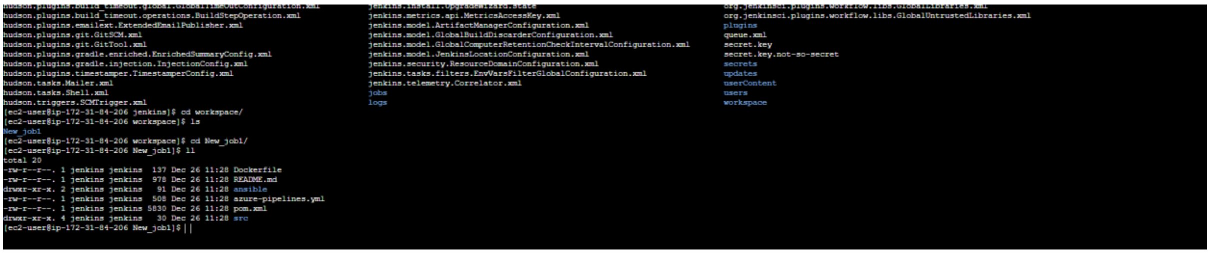
And then create a job





Stage:2





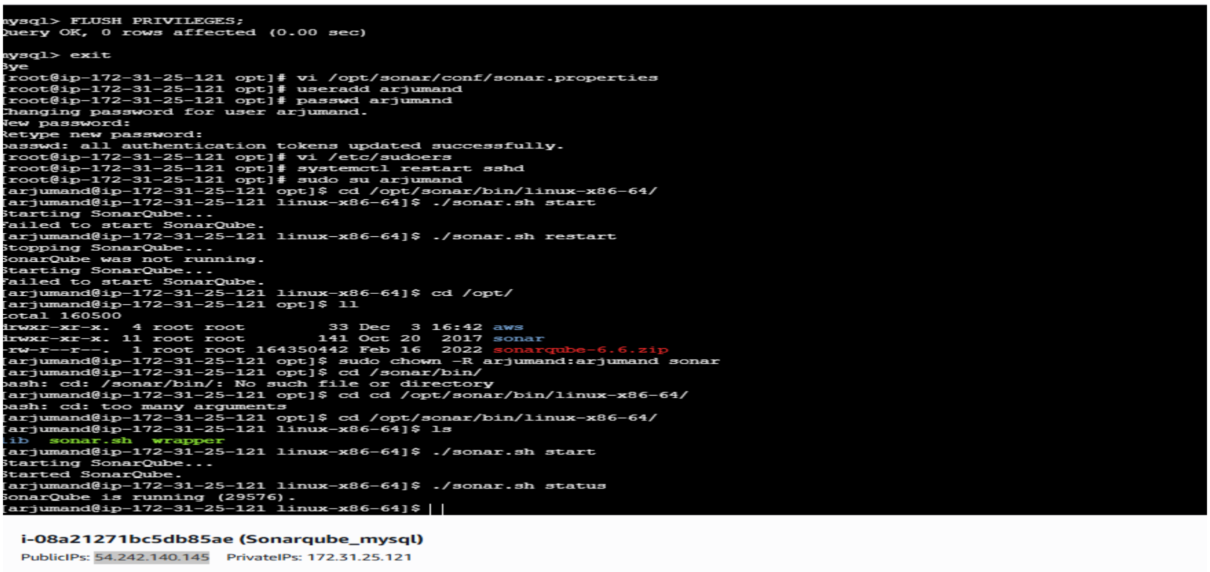
Installed SQL server

```
Downloading Packages:
(1/6): libxcrypt-compat-4.4.33-7.amzn2023.x86_64.rpm
(2/6): mysql-community-common-5.7.44-1.el7.x86_64.rpm
(3/6): ncurses-compat-libs-6.2-4.20200222.amzn2023.0.6.x86_64.rpm
(4/6): mysql-community-libs-5.7.44-1.el7.x86_64.rpm
(5/6): mysql-community-client-5.7.44-1.el7.x86_64.rpm
(6/6): mysql-community-server-5.7.44-1.el7.x86_64.rpm
--------------------------------------------------------------------------------
Total
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
  Preparing        :
  Installing       : mysql-community-common-5.7.44-1.el7.x86_64
  Installing       : mysql-community-libs-5.7.44-1.el7.x86_64
  Running scriptlet: mysql-community-libs-5.7.44-1.el7.x86_64
  Installing       : ncurses-compat-libs-6.2-4.20200222.amzn2023.0.6.x86_64
  Installing       : mysql-community-client-5.7.44-1.el7.x86_64
  Installing       : libxcrypt-compat-4.4.33-7.amzn2023.x86_64
  Running scriptlet: mysql-community-server-5.7.44-1.el7.x86_64
  Installing       : mysql-community-server-5.7.44-1.el7.x86_64
  Running scriptlet: mysql-community-server-5.7.44-1.el7.x86_64
/usr/lib/tmpfiles.d/mysql.conf:23: Line references path below legacy directory /var/run/, updating /var/run/mysqld → /run/mysqld; please update the tmpfiles.d/ drop-in file accordingly.

  Verifying        : libxcrypt-compat-4.4.33-7.amzn2023.x86_64
  Verifying        : ncurses-compat-libs-6.2-4.20200222.amzn2023.0.6.x86_64
  Verifying        : mysql-community-client-5.7.44-1.el7.x86_64
  Verifying        : mysql-community-common-5.7.44-1.el7.x86_64
  Verifying        : mysql-community-libs-5.7.44-1.el7.x86_64
  Verifying        : mysql-community-server-5.7.44-1.el7.x86_64

Installed:
  libxcrypt-compat-4.4.33-7.amzn2023.x86_64          mysql-community-client-5.7.44-1.el7.x86_64     mysql-community-common-5.7.44-1.el7.x86_64     mysql-community-libs-5.7.44-1.el7.x86_64     mysql-communi
  ncurses-compat-libs-6.2-4.20200222.amzn2023.0.6.x86_64

Complete!
[root@ip-172-31-84-206 ~]# sudo systemctl start mysqld.service
[root@ip-172-31-84-206 ~]# systemctl start mysqld
 systemctl enable mysqld
[root@ip-172-31-84-206 ~]# netstat -na | grep 3306
tcp6       0      0 :::3306                 :::*                    LISTEN
[root@ip-172-31-84-206 ~]# ||
```

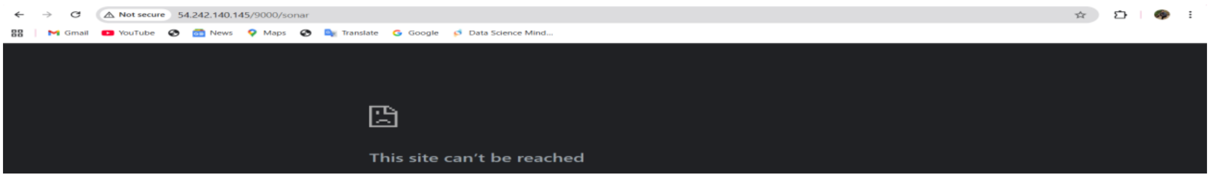
```
Complete!
[root@ip-172-31-84-206 ~]# sudo systemctl start mysqld.service
[root@ip-172-31-84-206 ~]# systemctl start mysqld
[root@ip-172-31-84-206 ~]# systemctl enable mysqld
[root@ip-172-31-84-206 ~]# netstat -na | grep 3306
[root@ip-172-31-84-206 ~]# systemctl status mysqld
● mysqld.service - MySQL Server
     Loaded: loaded (/usr/lib/systemd/system/mysqld.service; enabled; preset: disabled)
    Drop-In: /etc/systemd/system/mysqld.service.d
             └─override.conf
     Active: active (running) since Fri 2025-12-26 13:28:45 UTC; 1min 29s ago
       Docs: man:mysqld(8)
             http://dev.mysql.com/doc/refman/en/using-systemd.html
   Main PID: 40414 (mysqld)
     Status: "Server is operational"
      Tasks: 36 (limit: 9297)
     Memory: 558.9M
        CPU: 7.905s
     CGroup: /system.slice/mysqld.service
             └─40414 /usr/sbin/mysqld --skip-grant-tables --skip-networking

Dec 26 13:28:29 ip-172-31-84-206.ec2.internal systemd[1]: Starting mysqld.service - MySQL Server...
Dec 26 13:28:45 ip-172-31-84-206.ec2.internal systemd[1]: Started mysqld.service - MySQL Server.
[root@ip-172-31-84-206 ~]# grep 'temporary' /var/log/mysqld.log
[root@ip-172-31-84-206 ~]# mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 10
Server version: 8.0.44 MySQL Community Server - GPL

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

```
2 rows in set (0.00 sec)

mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| sys                |
+--------------------+
4 rows in set (0.00 sec)

mysql> CREATE DATABASE sonar CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
Query OK, 1 row affected (0.00 sec)

mysql>
mysql> GRANT ALL PRIVILEGES ON sonar.* TO 'sonar'@'localhost';
Query OK, 0 rows affected (0.00 sec)

mysql> GRANT ALL PRIVILEGES ON sonar.* TO 'sonar'@'%';
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.00 sec)

mysql> SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| sonar              |
| sys                |
+--------------------+
5 rows in set (0.00 sec)

mysql>
```

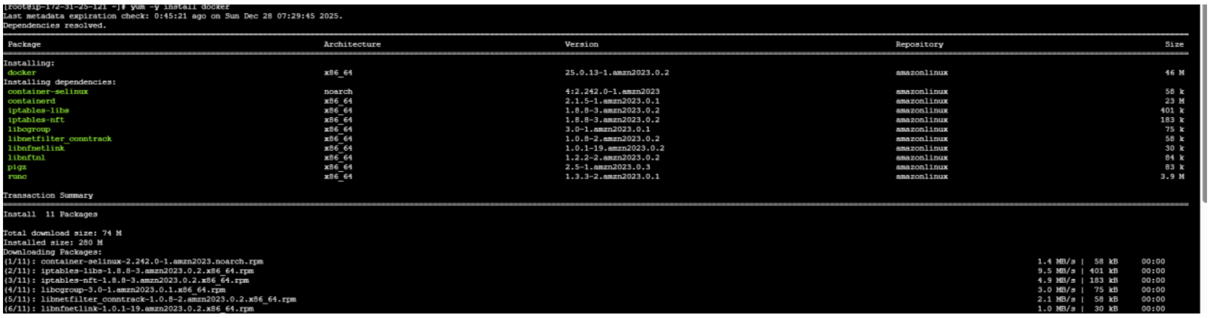
```
| information_schema |
| mysql              |
| performance_schema |
| sonar              |
| sys                |
+--------------------+
5 rows in set (0.00 sec)

mysql> CREATE USER sonar@localhost IDENTIFIED BY 'Sonar@123';
Query OK, 0 rows affected (0.00 sec)

mysql> CREATE USER sonar@'%' IDENTIFIED BY 'Sonar@123';
Query OK, 0 rows affected (0.00 sec)

mysql> GRANT ALL ON sonar.* TO sonar@localhost;
Query OK, 0 rows affected (0.00 sec)

mysql> GRANT ALL ON sonar.* TO sonar@'%';
Query OK, 0 rows affected (0.00 sec)

mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| sonar              |
| sys                |
+--------------------+
5 rows in set (0.00 sec)

mysql> SELECT User FROM mysql.user;
+---------------+
| User          |
+---------------+
| sonar         |
| mysql.session |
| mysql.sys     |
| root          |
| sonar         |
+---------------+
5 rows in set (0.00 sec)
```



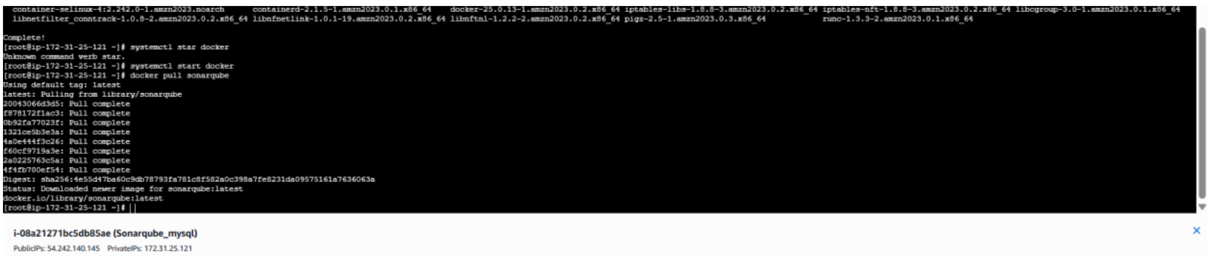
MySQL is not supporting sonarqube it is officially ended



Docker instalation



Docker pull sonarqube

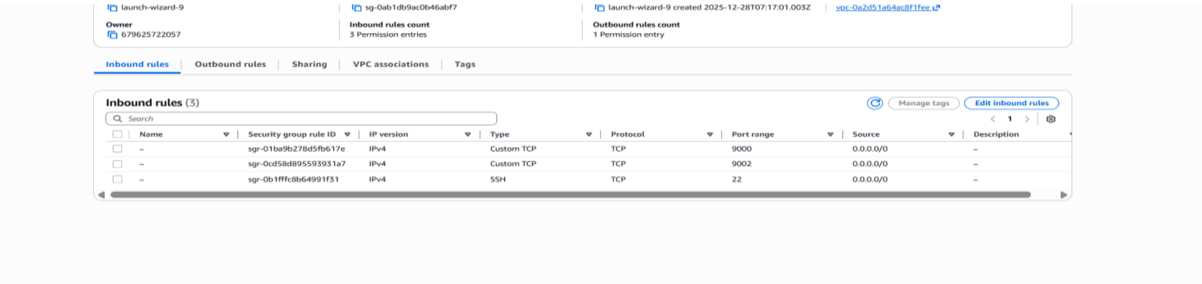


Docker images

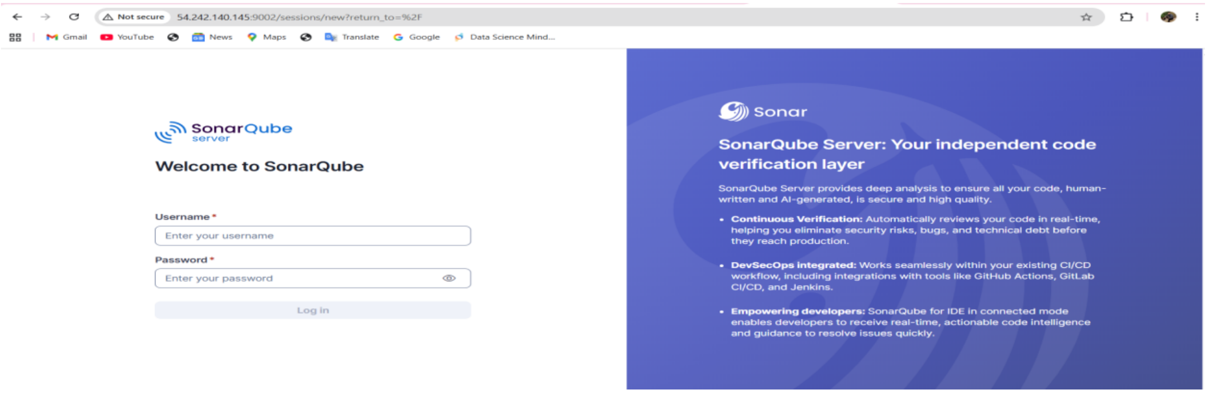
```
4f4fb700ef54: Pull complete
Digest: sha256:4e55d47ba60c9db78793fa781c8f582a0c398a7fe8231da09575161a7636063a
Status: Downloaded newer image for sonarqube:latest
docker.io/library/sonarqube:latest
[root@ip-172-31-25-121 ~]# docker images
REPOSITORY   TAG      IMAGE ID       CREATED       SIZE
sonarqube    latest   247ce055a80b   2 weeks ago   1.27GB
[root@ip-172-31-25-121 ~]#
```

i-08a21271bc5db85ae (Sonarqube_mysql)
PublicIPs: 54.242.140.145 PrivateIPs: 172.31.25.121

allowed 9002 port number in security group

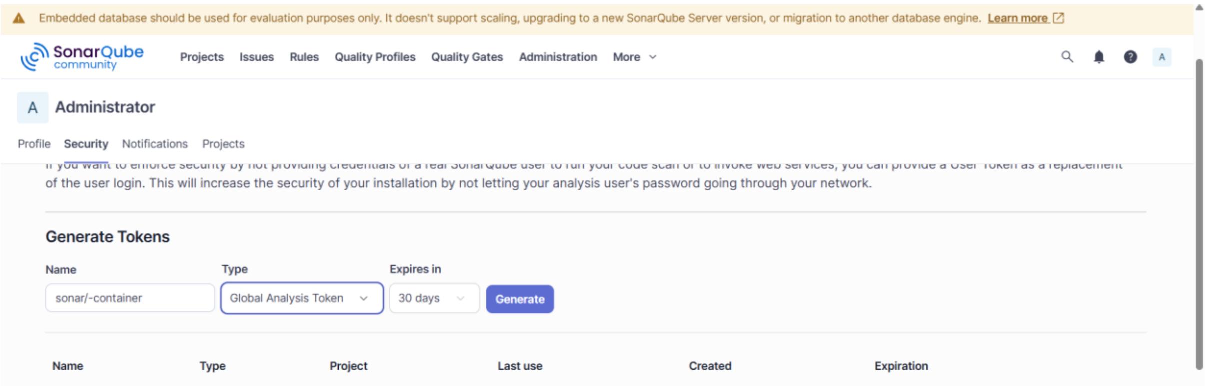


Sonarqube is in deployed successfully

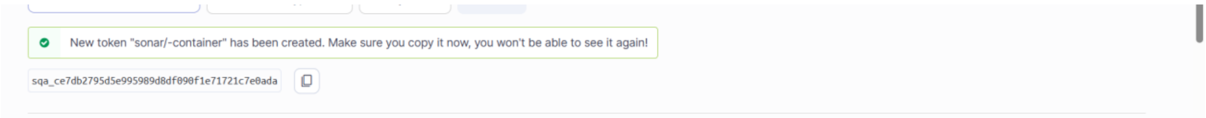


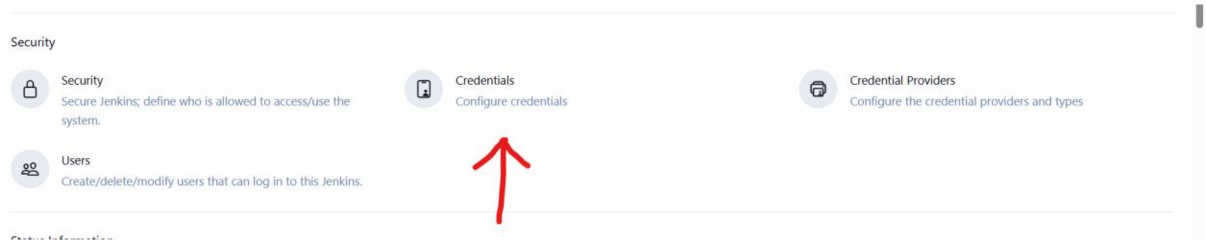

```
#Configure here general information about the environment, such as SonarQube server connection details for example
#No information about specific project should appear here

#----- Default SonarQube server
sonar.host.url=http://http://54.242.140.145/:9002

#----- Default source code encoding
#sonar.sourceEncoding=UTF-8
~
~
~
~
~
~
~
```

```
Last login: Sun Dec 28 08:59:25 2025 from 18.206.107.27
[ec2-user@ip-172-31-21-117 ~]$ sudo su -
Last login: Sun Dec 28 08:39:47 UTC 2025 on pts/1
[root@ip-172-31-21-117 ~]# cd /opt/sonar_scanner
[root@ip-172-31-21-117 sonar_scanner]# cd conf/
[root@ip-172-31-21-117 conf]# ls
sonar-scanner.properties
[root@ip-172-31-21-117 conf]# vi sonar-scanner.properties
[root@ip-172-31-21-117 conf]#
```

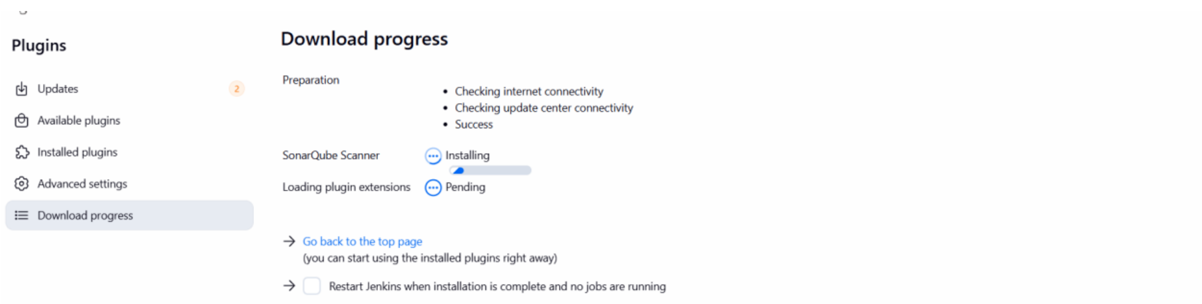


Generated key

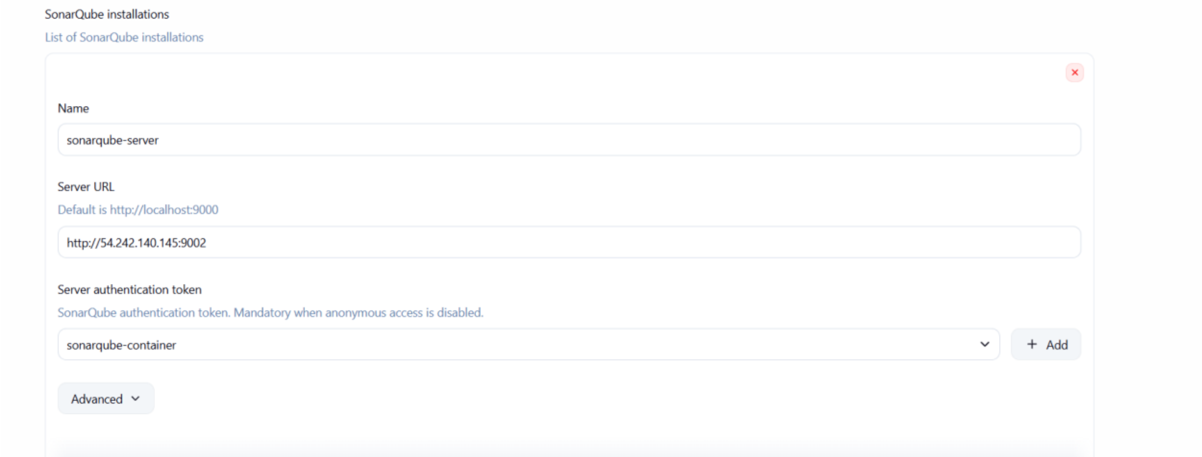




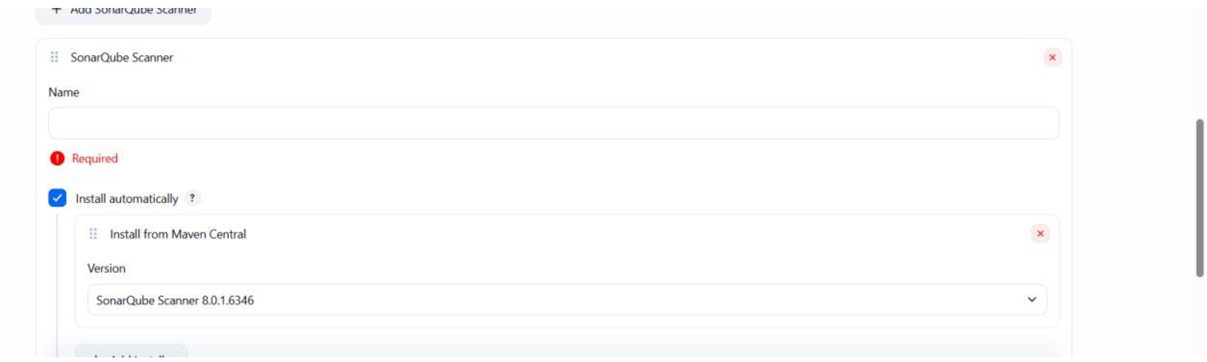
Install sonaqube package in plugin



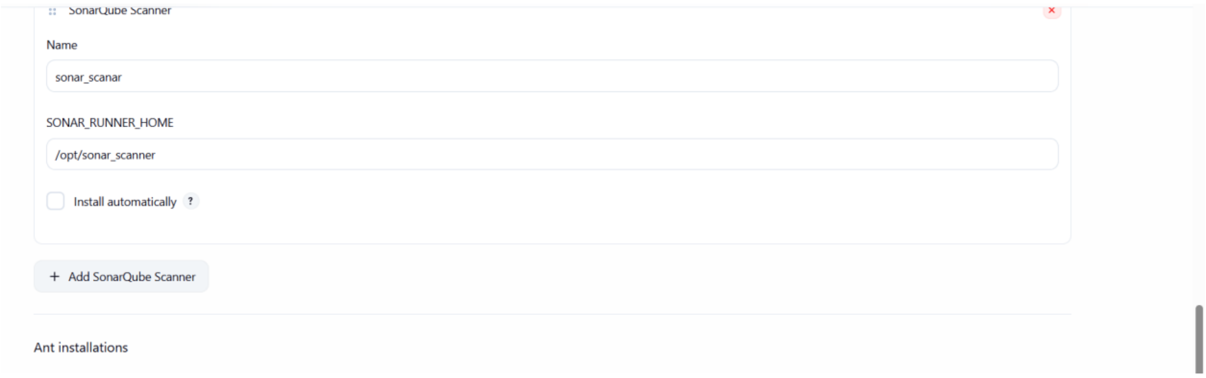
To connect to sonarqube I have connected with system configuration added sonarqube credentials

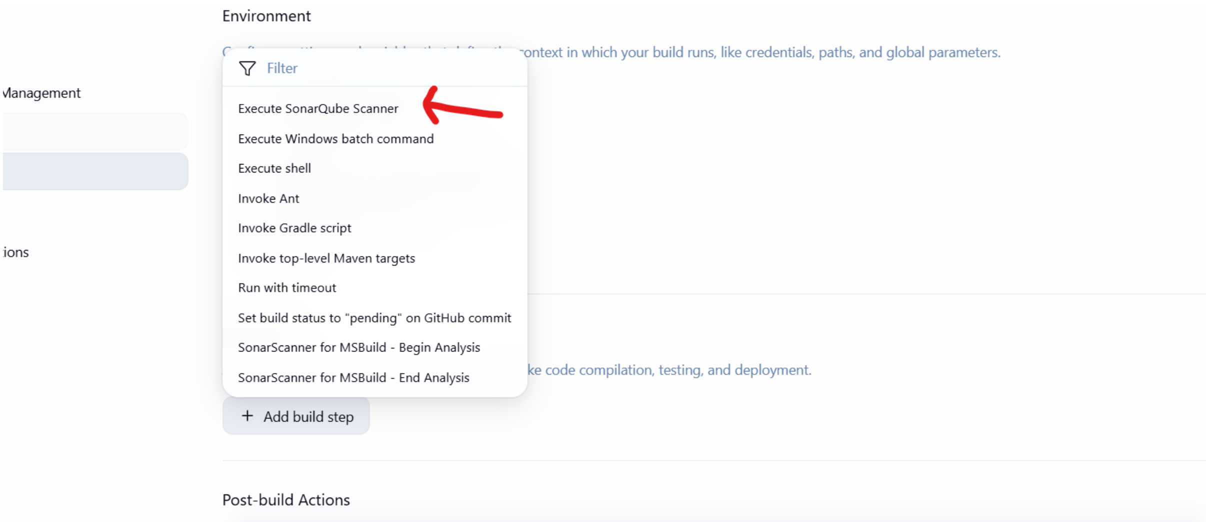


Go to manage Jenkins > tools > add sonar scanner

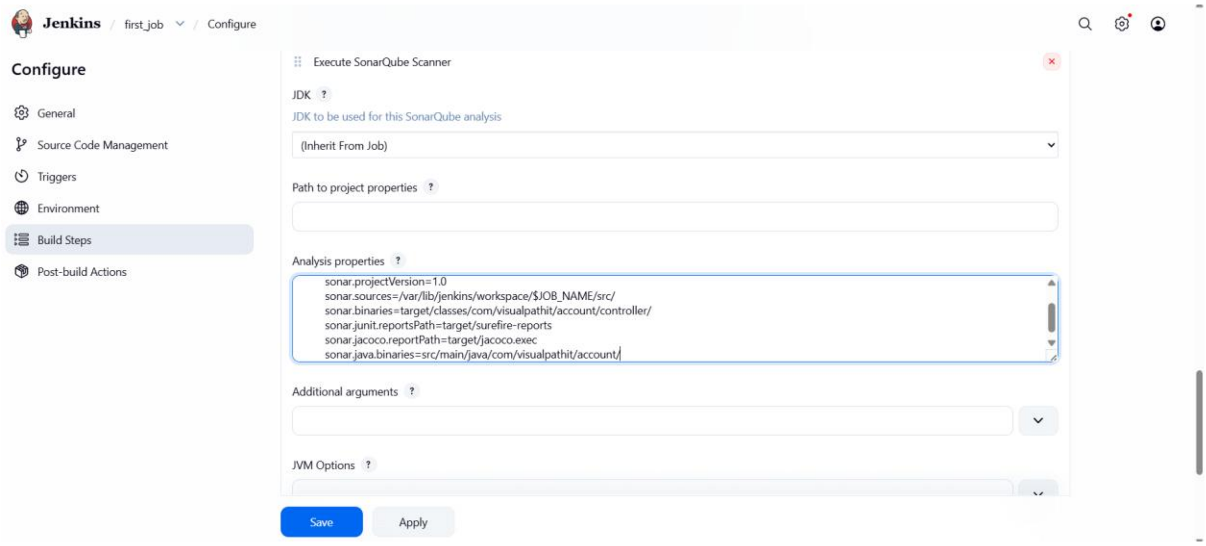


Find the location of the sonar scanner

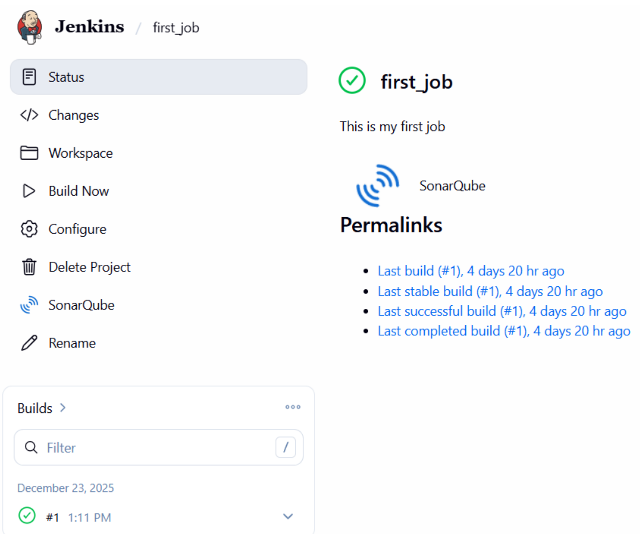




Make changes in the job and add properties in SonarQube section

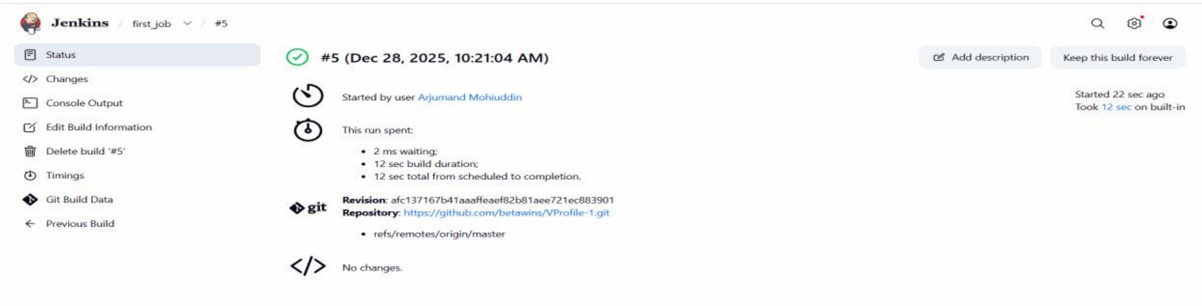


Logo was appeared as sonar

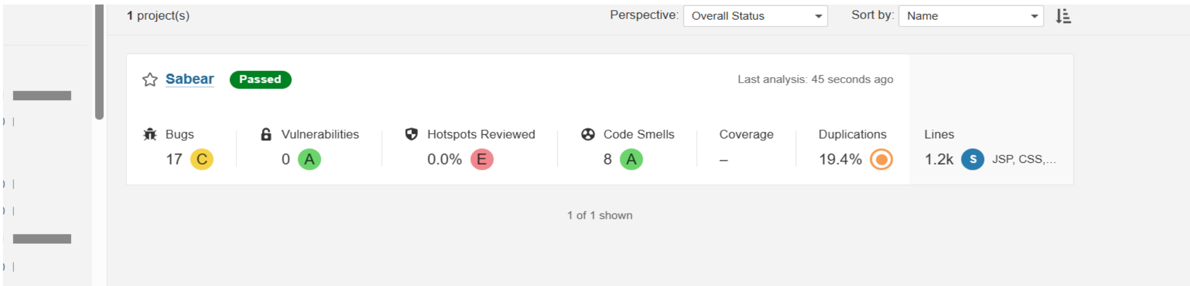


Job was executed and successfully it is updated in sonarqube server

Job is executed successfully

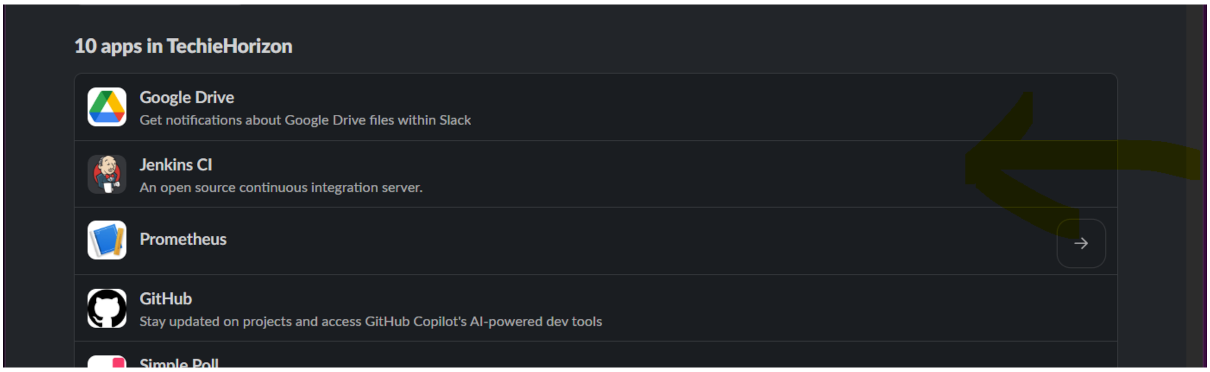


Code is successfully send to sonarqube and tested the quality of the code

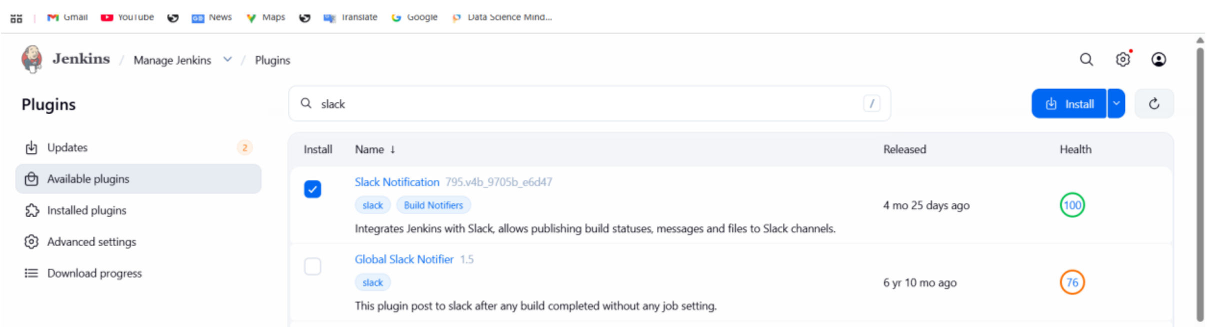


Stage 3 Configure with slag

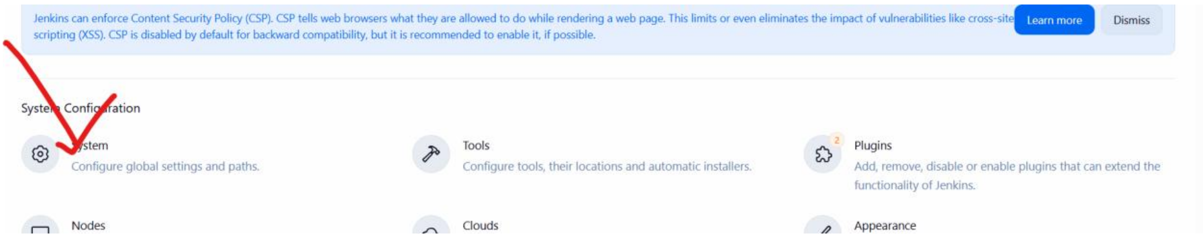
Go to slack click create new and select app select Jenkins cl



Go to manage Jenkins select plugin and select slack notification then install it



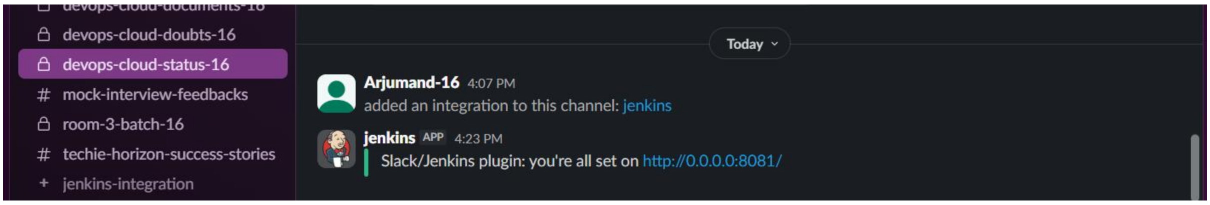
Go to manage Jenkins > system



Select slack selection add name of the slack organisation and token number provided by slack which will be added in credentials.

After adding credentials go for test connection

Slack is integrated sucessfully



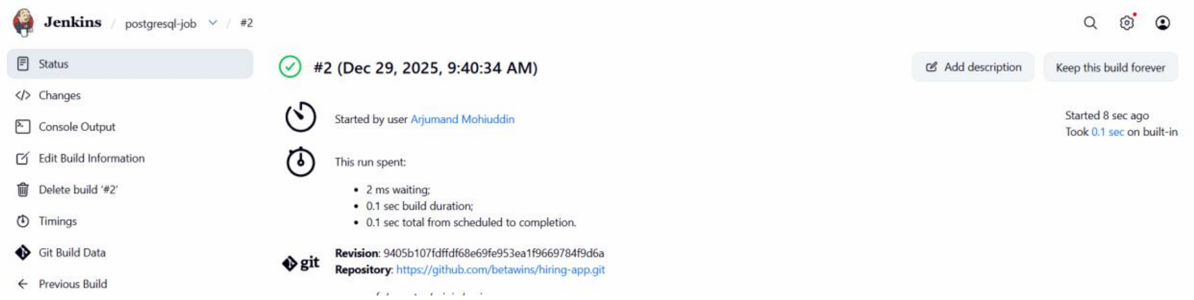
Task 2

Create one Jenkins job using the below code and create three stages:

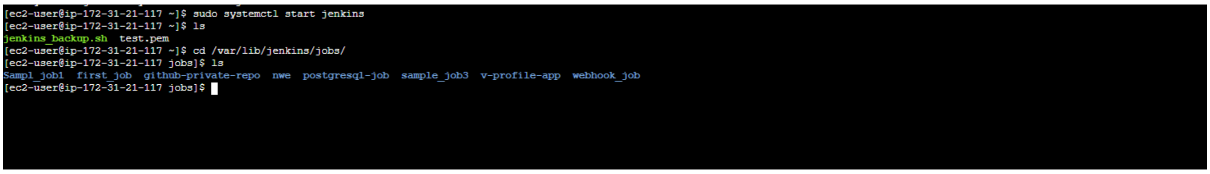
- **stage1: Git clone to download the source code.**
- **stage2: Sonarqube Integration to check the quality of code.**
- **stage3: Slack Integration to send the alerts to slack.**

Clone was done using new url- https://github.com/betawins/hiring-app.git with the main branch, instead of master.

- ## stage1: Git clone to download the source code.

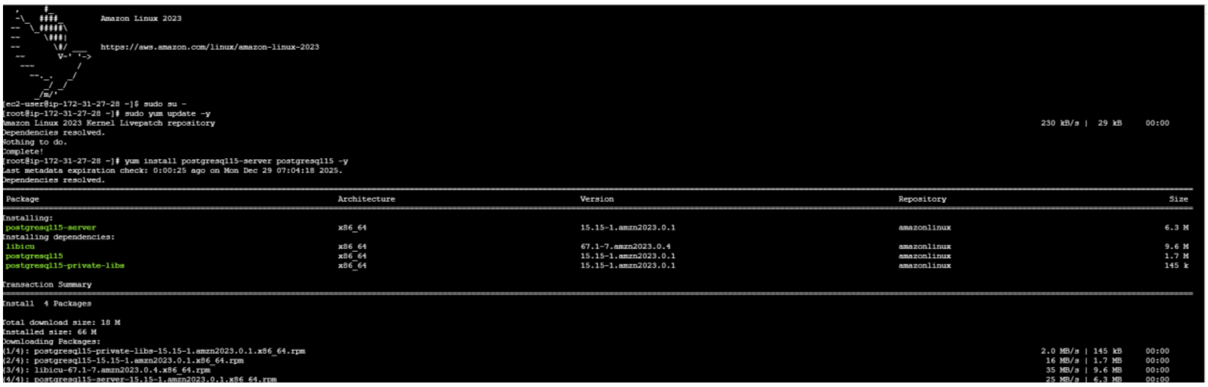


Clone is successful as we can see in the terminal as well



Stage2: Sonarqube Integration to check the quality of code.

Installed new server for sonarqube using postgresql using command 'yum install postgresql115-server postgresql115 -y'



sudo /usr/bin/postgresql-setup--initdb--unit postgresql

Now enable and start PostgreSQL:

bash

Copy code

sudo systemctl enable postgresql

sudo systemctl start postgresql

Check status:

bash

Copy code

sudo systemctl status postgresql

🧪 Check PostgreSQL Login

Bash

Copy code

sudo-u postgres psql

Run these commands one-by-one inside the PostgreSQL prompt:

CREATE DATABASE sonarqube;

CREATE USER sonar WITH ENCRYPTED PASSWORD 'StrongPassword123';

GRANT ALL PRIVILEGES ON DATABASE sonarqube TO sonar;

ALTER USER sonar WITH SUPERUSER;

(You can replace password with your own secure one)

Exit PostgreSQL:

\q

Configure PostgreSQL to Allow Local Authentication

Open pg_hba.conf:

sudo nano /var/lib/pgsql/data/pg_hba.conf

Change these lines (at bottom usually):

IPv4 local connections:

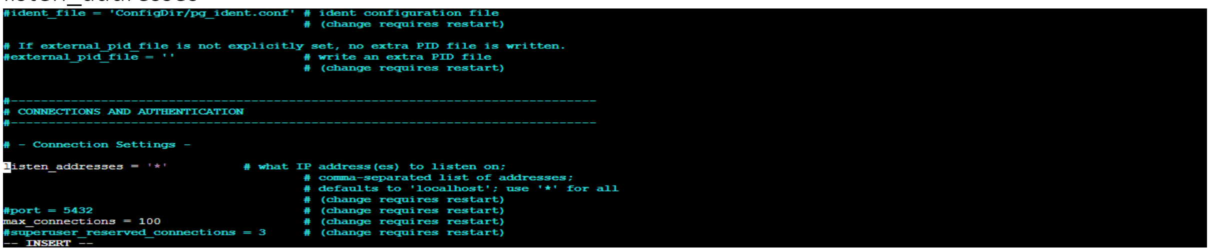
host   all   all   127.0.0.1/32   md5

Allow remote access:

sudo vi /var/lib/pgsql/15/data/postgresql.conf

Update:

listen_addresses='*'



Installed java using command 'sudo dnf install java-17-amazon-corretto -y'

- cd /opt
- sudo wget https://binaries.sonarsource.com/Distribution/sonarqube/sonarqube-9.9.1.69595.zip
- sudo yum install unzip -y
- sudo unzip sonarqube-9.9.1.69595.zip
- sudo mv sonarqube-9.9.1.69595 sonarqube

- sudo groupadd sonar
- sudo useradd -d /opt/sonarqube -g sonar sonar

- sudo chown -R sonar:sonar /opt/sonarqube

```
[root@ip-172-31-27-28 opt]# sudo groupadd sonar
sudo useradd -d /opt/sonarqube -g sonar sonar
sudo chown -R sonar:sonar /opt/sonarqube
useradd: warning: the home directory /opt/sonarqube already exists.
useradd: Not copying any file from skel directory into it.
[root@ip-172-31-27-28 opt]# ll
total 286988
drwxr-xr-x.  4 root  root         33 Dec  3 16:42 aws
drwxr-xr-x. 11 sonar sonar       172 Apr 27  2023 sonarqube
-rw-r--r--.  1 root  root  293875683 Apr 28  2023 sonarqube-9.9.1.69595.zip
[root@ip-172-31-27-28 opt]# |
```

Edit config:

sudo nano /opt/sonarqube/conf/sonar.properties

Set:

sonar.jdbc.username=sonar

sonar.jdbc.password=StrongPass123

sonar.jdbc.url=jdbc:postgresql://localhost:5432/sonarqube

Enable:

sonar.web.port=9000

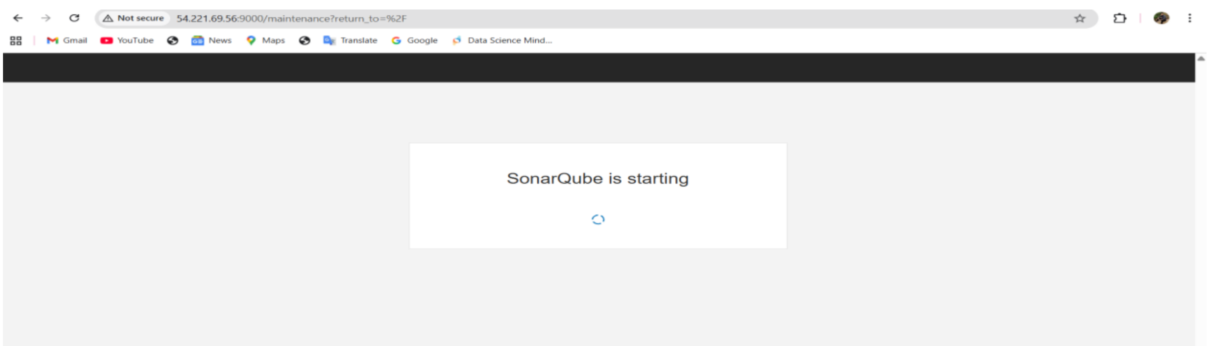
Save and close: CTRL+O → ENTER → CTRL+X

```
# User credentials.
# Permissions to create tables, indices and triggers must be granted to JDBC user.
# The schema must be created first.
sonar.jdbc.username=sonar
sonar.jdbc.password=Sonar123

#----- Embedded Database (default)
# H2 embedded database server listening port, defaults to 9092
```

```
#----- PostgreSQL 11 or greater
# By default the schema named "public" is used. It can be overridden with the parameter "currentSchema".
sonar.jdbc.url=jdbc:postgresql://localhost:5432/sonarqube
```

```
sonar.web.host=0.0.0.0

# Web context. When set, it must start with forward slash (for example /sonarqube).
# Changing this value and restarting the server can have unexpected consequences for logged in users,
# as they may end up with multiple conflicting browser cookies. If your users experience odd
# behaviors after you changed this value and restarted the server, ask them to clear their browser's
# cookies and log in again.
# The default value is root context (empty value).
#sonar.web.context=
# TCP port for incoming HTTP connections. Default value is 9000.
sonar.web.port=9000
```

Now port is working on 9000 allowed in SG



<u>ERROR:</u> I was unable to start sonarqube and deploy on website, I was getting This error after the setup of the steps:

Since SonarQube is now **running successfully**, here is a clean and professional summary of the **exact troubleshooting steps** I performed.

🛠️ SonarQube Deployment Troubleshooting – Summary

While deploying SonarQube on EC2 with PostgreSQL, SonarQube initially failed to start and Web UI was not accessible. Here are the steps taken to fix the issues:

1️⃣ Fix Linux Kernel Limits for Elasticsearch

SonarQube requires higher vm.max_map_count for Elasticsearch:

sudo sysctl-w vm.max_map_count=262144

To make it permanent:

echo "vm.max_map_count=262144" | sudo tee-a /etc/sysctl.conf

sudo sysctl-p

2️⃣ Set Correct Ownership and Permissions

Elasticsearch cannot run as root. So we created sonar user and assigned permissions:

sudo useradd-d /opt/sonarqube-g sonar sonar

sudo chown-R sonar:sonar /opt/sonarqube

sudo chmod-R 775 /opt/sonarqube

Then started SonarQube as sonar user:

sudo su- sonar

cd /opt/sonarqube/bin/linux-x86_64/

./sonar.sh start

3️⃣ Correct PostgreSQL Authentication Errors

SonarQube logs showed:

FATAL: Ident authentication failed for user "sonar"

So I updated PostgreSQL authentication mode from ident to md5:

Edited file:

/var/lib/pgsql/15/data/pg_hba.conf

Updated rules:

host all all 127.0.0.1/32 md5

host all all ::1/128 md5

Restarted PostgreSQL:

sudo systemctl restart postgresql

4️⃣ Created SonarQube Database and Password

Logged into PostgreSQL:

sudo su- postgres

psql

Executed:

ALTER USER sonar WITH PASSWORD 'Sonar123';

CREATE DATABASE sonarqube OWNER sonar;

\q

Tested connection:

psql-U sonar-d sonarqube-h 127.0.0.1-W

5️⃣ Updated SonarQube Configuration

Edited:

/opt/sonarqube/conf/sonar.properties

Configured database URL and credentials:

sonar.jdbc.username=sonar

sonar.jdbc.password=Sonar123

sonar.jdbc.url=jdbc:postgresql://localhost:5432/sonarqube

6️⃣ Opened Required Port in AWS

Inbound rules updated for EC2 security group:

Port	Protocol	Source
9000	TCP	0.0.0.0/0
22	TCP	0.0.0.0/0

7️⃣ Restarted SonarQube & Verified

sudo su- sonar

cd /opt/sonarqube/bin/linux-x86_64/

./sonar.sh stop

./sonar.sh start

./sonar.sh status

Checked logs until no errors:

tail-f /opt/sonarqube/logs/sonar.log

🔍 Final Result

SonarQube is successfully deployed and accessible at:

http://<Public-IP>:9000

Login:

username: admin

password: admin

🎯 Conclusion

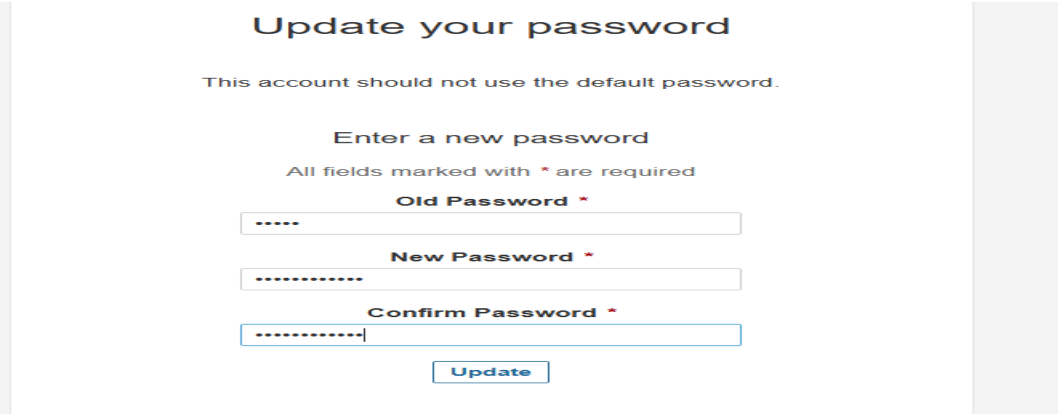
By fixing:
✔ Kernel settings
✔ File permissions
✔ PostgreSQL authentication

✓ Database configuration
✓ Security group access

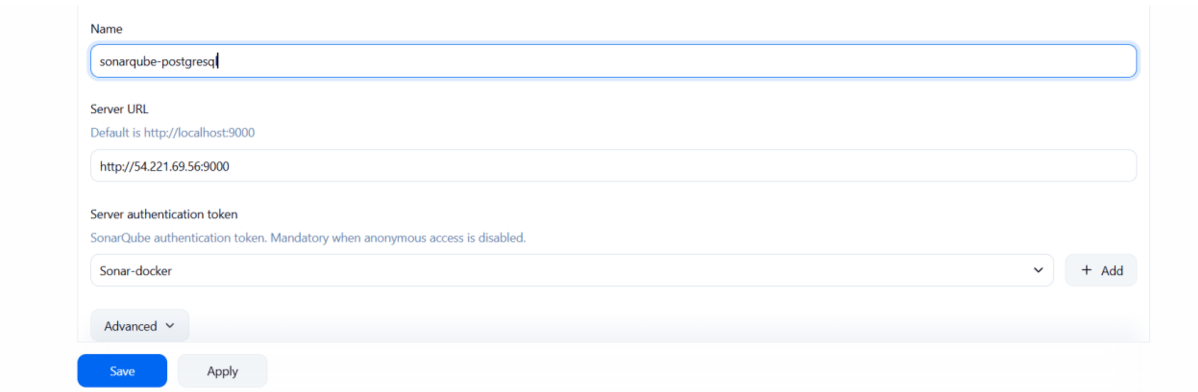
SonarQube started successfully and UI became accessible.

Would you like me to create your Jenkins stages now? 🚀

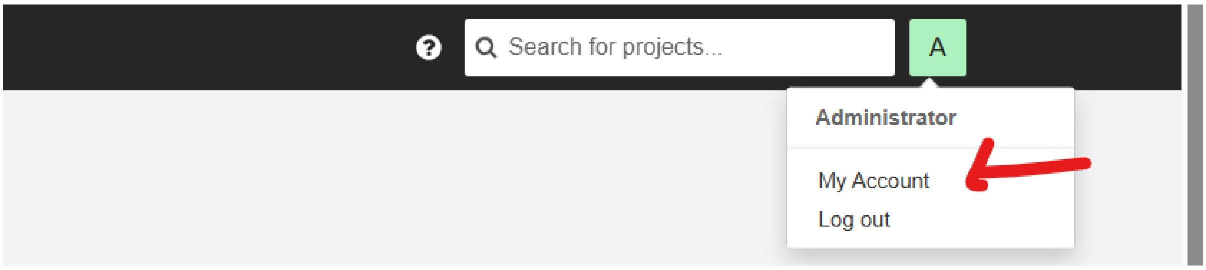
Updated password:



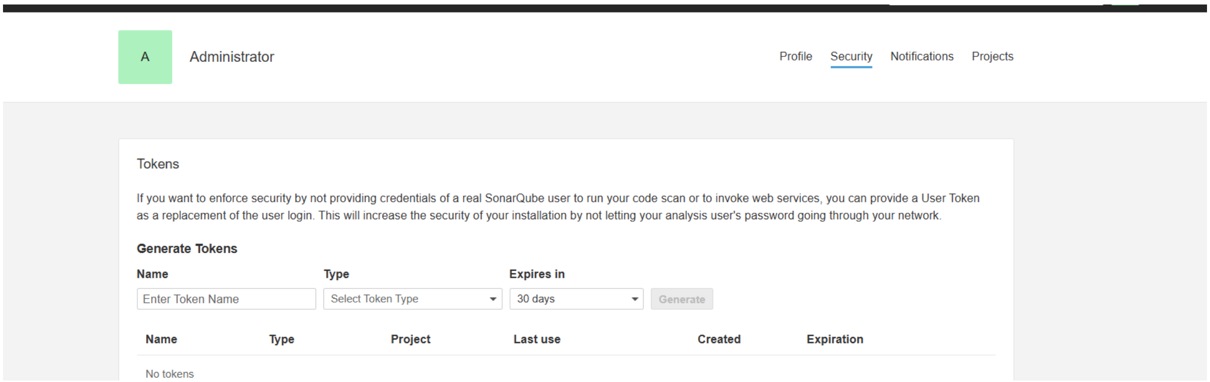
In Jenkins I have added new url under sonarqube installation



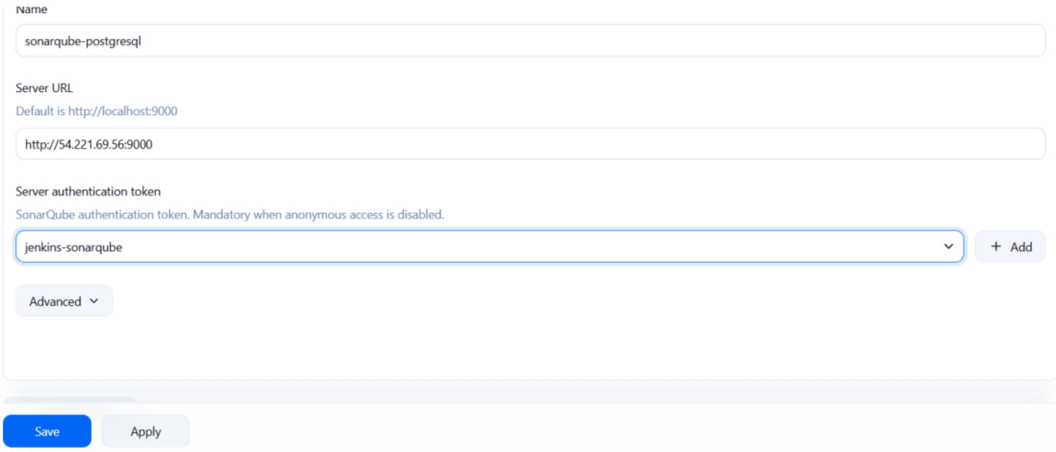
Go to Soanrqube server and go to logo and choose my account and the go to security tab



Enter the details in security tab and create token which will be added in system configuration in jenkins

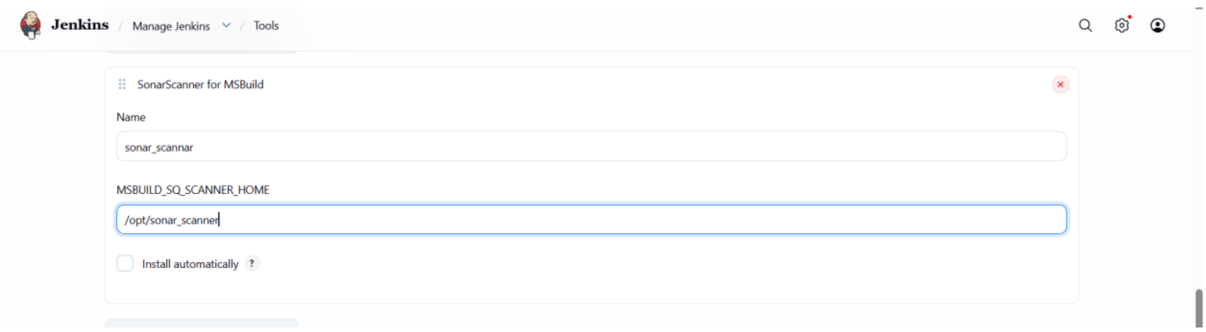


Add sonarqube token in credentials in secret text option. And provide name and description

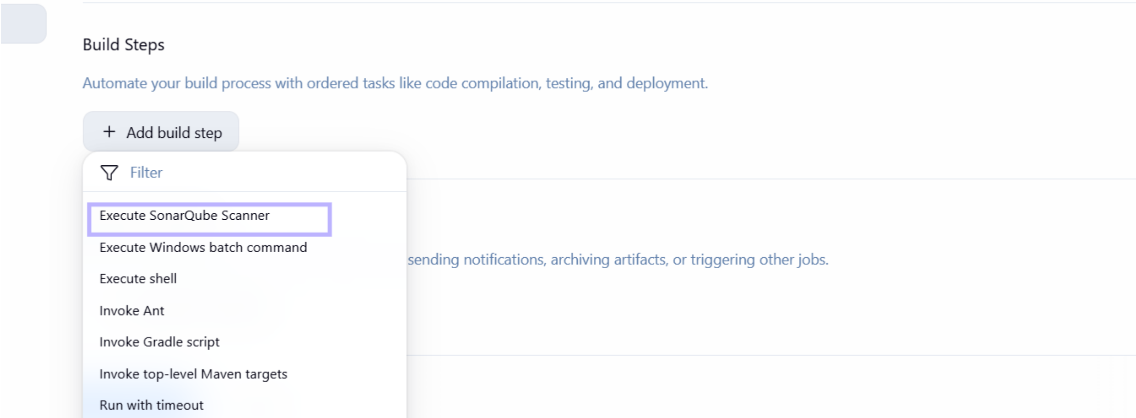


In Jenkins server add new url of sonarqube url after installing in opt, path will be /opt/sonar_scannar/conf/sonar-scannar.properties

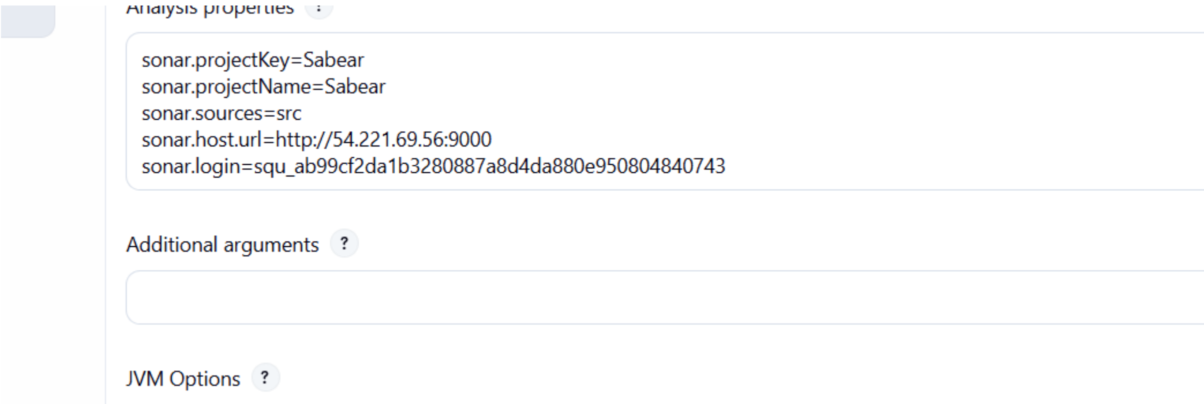
Go to tools in Jenkins and pass the sonar scanner path and name



Go to jobs and buit steps add

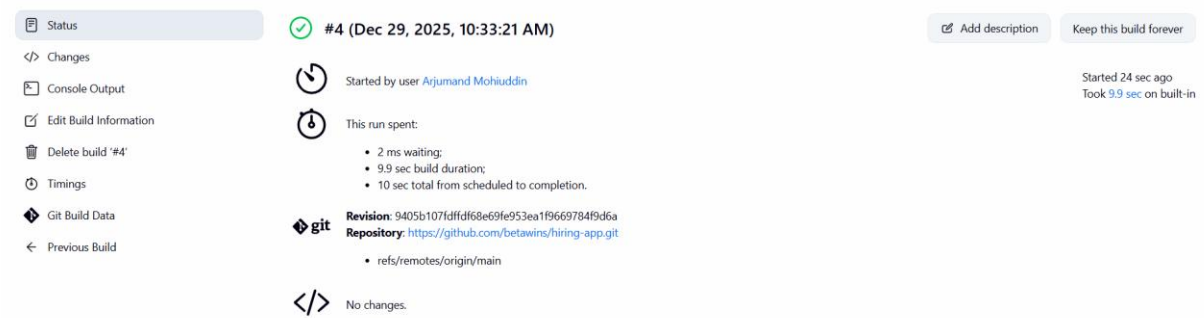


Add details of the project properties along with the token and URL of the sonarqube server

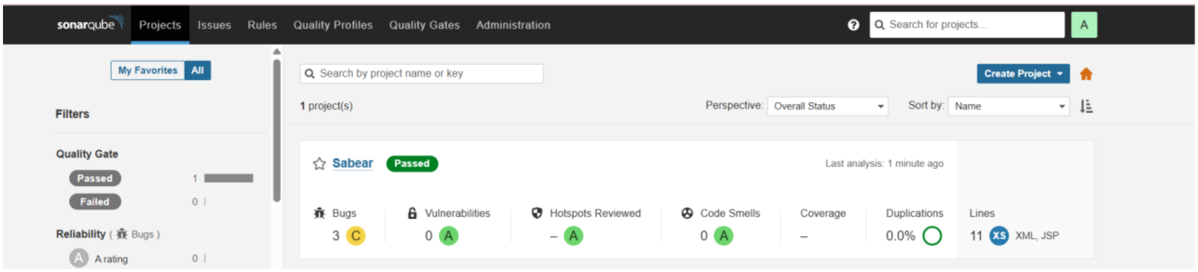


Afer changing the configuration of the job, Sonarqube logo will be appeared, next step will be testing the job the job

Sonarqube job is executed sucessfully

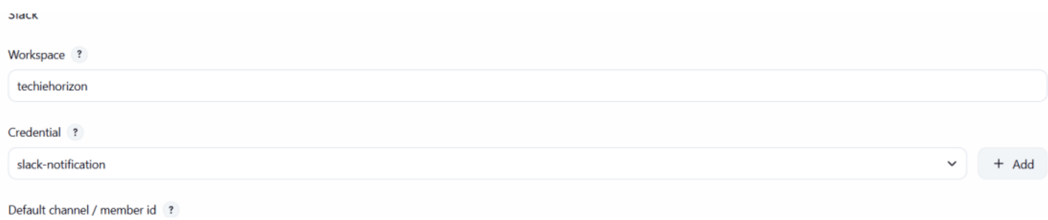


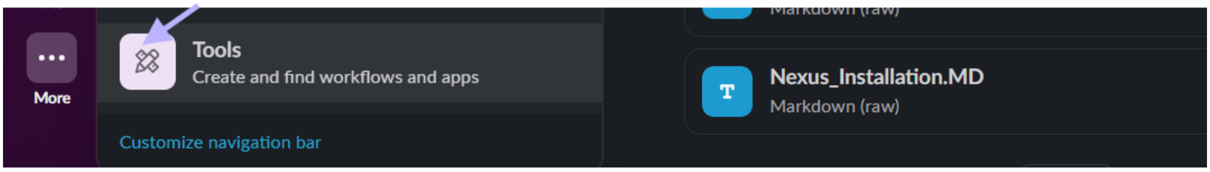
As we can see code quality is passed in sonarqube



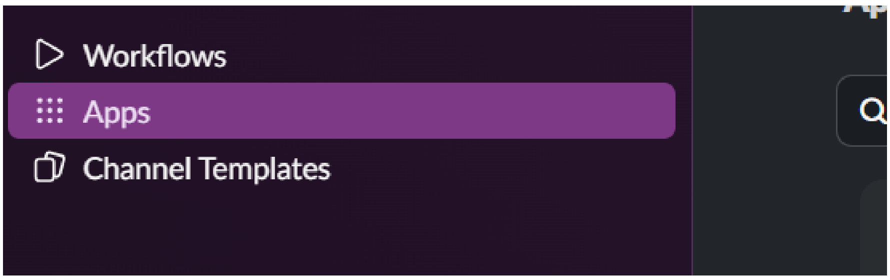
Stage3: Slack Integration to send the alerts to slack.

Go to tools in slack and click tools

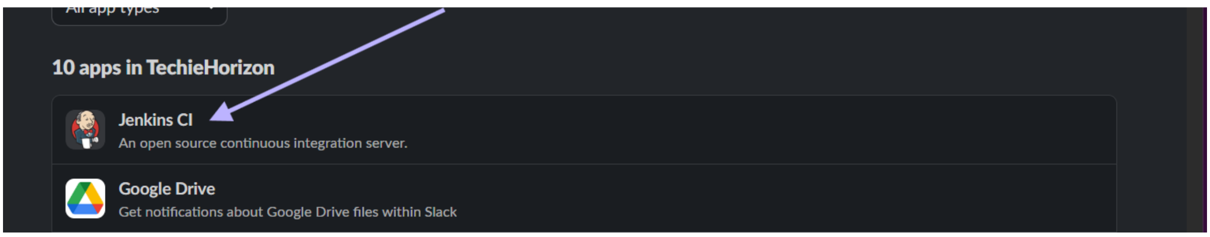




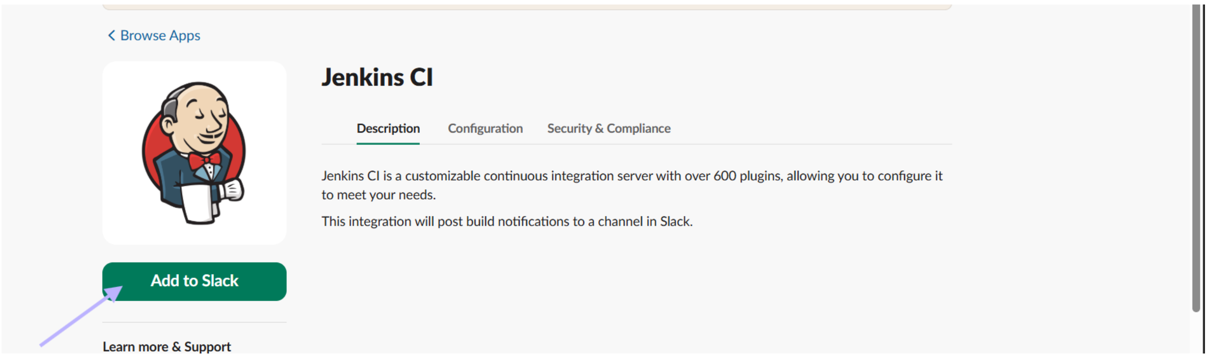
Select apps



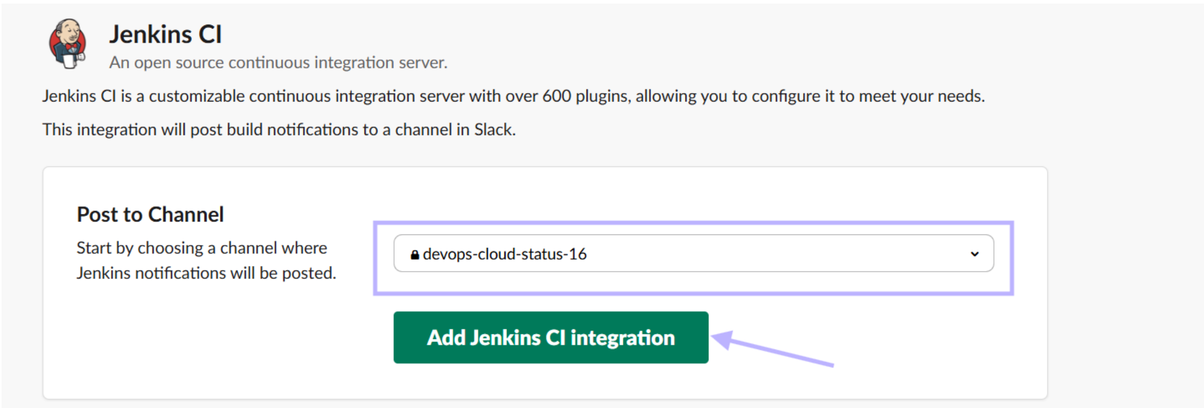
Select Jenkins CI



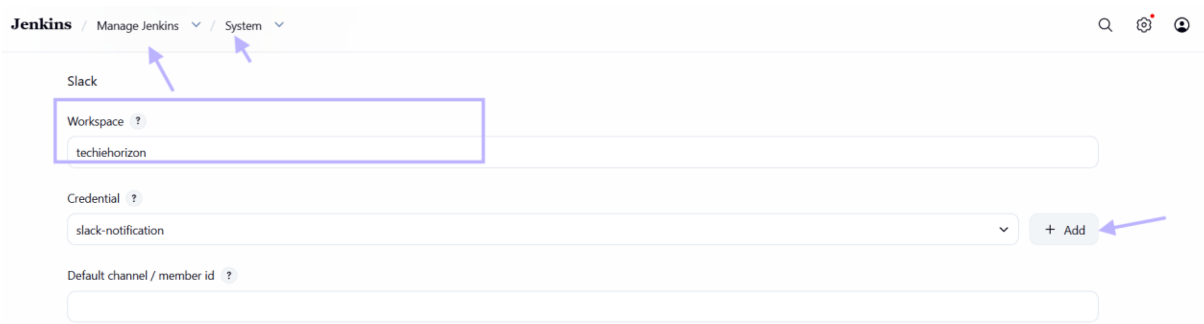
ADD Jenkins CI notifications



Select the channel

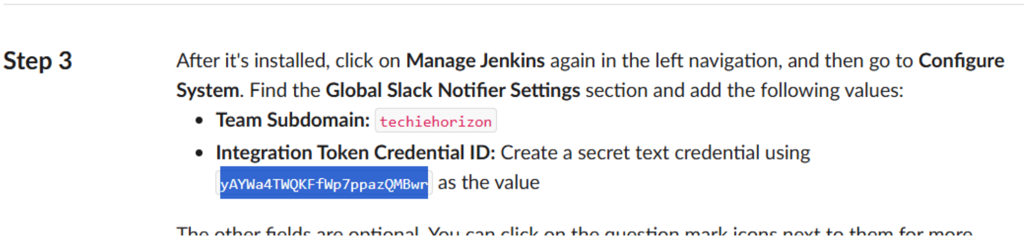


Go to Jenkins manage Jenkins > system configuration > select slack write the name of the organisation then click add credentials



And slect secret text

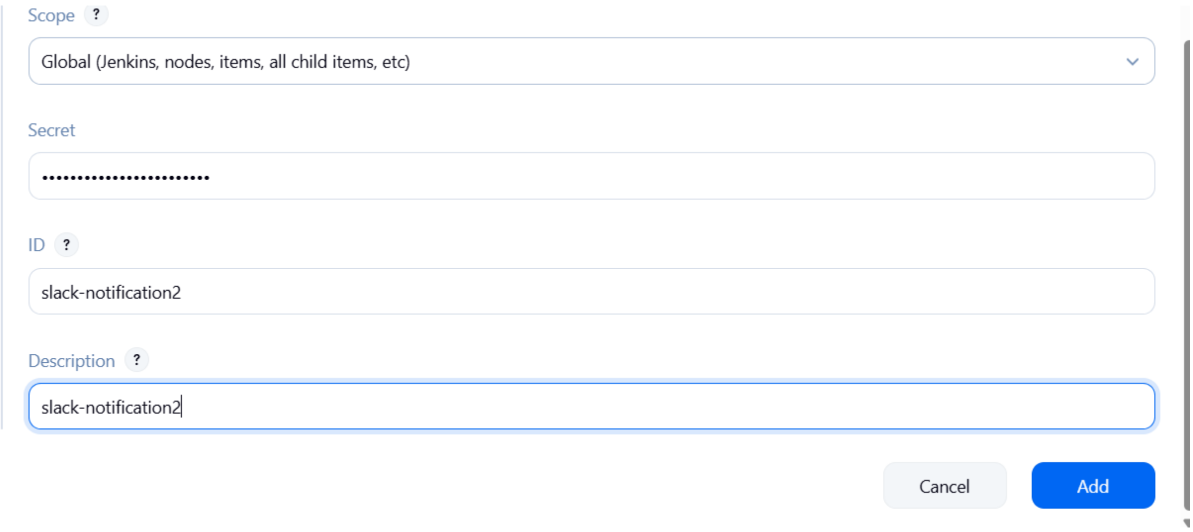
Select token from slacks on step 3


Step 3

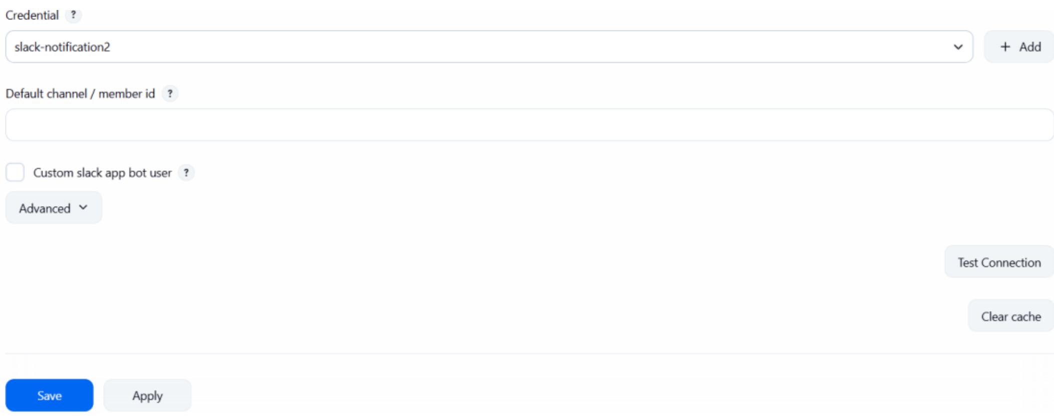
After it's installed, click on **Manage Jenkins** again in the left navigation, and then go to **Configure System**. Find the **Global Slack Notifier Settings** section and add the following values:

- **Team Subdomain:** `techiehorizon`
- **Integration Token Credential ID:** Create a secret text credential using `yAYWa4TWQKFfWp7ppazQMBwr` as the value

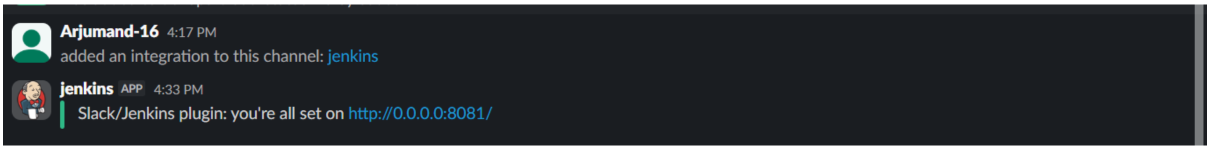
Paste the token and prove ID and description



After adding test connection then save



Tested sucessfully



Error 1: Branch Not Found

• Issue: Jenkins build failed with “Couldn't find any revision to build”

• Cause: Git repository used main branch instead of master

• Fix: Updated branch specifier in Jenkins from */master to */main

Error 2: SonarQube Authentication Failure

• Issue: Jenkins could not authenticate with SonarQube

• Cause: SonarQube requires token-based authentication

• Fix: Generated SonarQube token and configured it as a Jenkins Secret

Text credential

Error 3: Java Version Compatibility

• Issue: SonarQube Scanner requires Java 17

• Cause: Default Java version was incompatible

• Fix: Explicitly configured JAVA_HOME and PATH to use Amazon Corretto Java 17 in Jenkins

build step

Error 4: No Build Notifications

• Fix: Explicitly configured JAVA_HOME and PATH to use Amazon Corretto Java 17 in Jenkins

build step

Error 4: No Build Notifications

• Issue: No visibility of build results

• Cause: Notifications not configured

• Fix: Configured Slack Notifications under Jenkins Post-build Actions

Conclusion

A complete CI pipeline was successfully implemented using Jenkins integrated with SonarQube and

GitHub. SonarQube was deployed using Docker, enabling scalable and isolated code quality analysis.

Jenkins was configured to perform automated code scanning during builds, and Quality Gate results were validated successfully. Slack notifications were enabled to provide real-time build status updates. This setup ensures improved code quality, faster feedback, and better collaboration within the development workflow